MiniSpotify Design

Assignment #3 | ECE 422C | Java 17 | Revised October 14, 2026

Purpose and compatibility

The supplied MiniSpotify 0.8 project is refactored into a scalable, in-memory backend. The package remains **edu.utexas.ece422c.minispotify**. The MusicService interface is unchanged, MusicServices.create() still returns LegacyMusicService, and the frozen Song constructor/getters and Playlist name/owner getters retain their signatures. Maven dependencies remain limited to the supplied JUnit 5 dependency.

Changes from the inherited implementation

Linear catalog and user scans are replaced by hash-based lookup. Artist and genre indexes preserve catalog insertion order within each bucket. Favorites, follows, collaborator permissions, and lifetime listening membership use sets. Playlist contents remain ordered lists that permit repeated songs. History changes from an unbounded list to a deque containing at most 50 play events.

Missing collaborative, follow, and recommendation operations are implemented. Song duration must be positive, arguments are checked consistently, ranking ties use case-insensitive titles then case-sensitive song IDs, and returned collections protect the service state. Playlist mutators are removed from the client-facing class; permission-checked edits use a separate internal PlaylistState.

Responsibilities and code navigation

Class	Responsibility
LegacyMusicService	Implements the frozen API; resolves users and songs before delegating updates
SongCatalog	Owns catalog identity, insertion order, case folding, and artist/genre indexes
SongRanking	Owns global play counts and a shared top-n selection algorithm
User	Owns playlists, follows, favorites, the recent deque, and lifetime played IDs
PlaylistState / Playlist	Internal editable playlist with permissions / immutable client snapshot
Song / Validation	Immutable ID-based song value / shared null and nonnegative checks
CatalogLoader / MiniSpotifyApp	Retained CSV loader with row errors / expanded runnable starter demo

The implementation follows the requested comment and brace style. These classes divide the solution by state and responsibility so that collection invariants can be explained and maintained locally.

Collections and indexes

Part 1 of 2 | catalog, account lookup, and playlist state

Field or index	Declared type	Concrete collection used	Reason for the choice	Expected complexity of important operations
SongCatalog songsById	Map<String, Song>	LinkedHashMap	Expected constant-time ID lookup plus catalog insertion-order traversal; IDs stay case-sensitive	get / contains / put: expected O(1); ordered traversal: O(C)
SongCatalog songsByArtist	Map<String, List<Song>>	HashMap containing ArrayList buckets	Case-folded artist keys select only matching songs; appends preserve insertion order	bucket lookup: expected O(1); append: amortized O(1); returned k-song snapshot: O(k)
SongCatalog songsByGenre	Map<String, List<Song>>	HashMap containing ArrayList buckets	Indexed exact genre search; recommendations reuse only relevant genre buckets	lookup: expected O(1); append: amortized O(1); returned k-song snapshot: O(k)
LegacyMusicService users	Map<String, User>	HashMap	Unique, case-sensitive usernames with direct account lookup	create / find / duplicate check: expected O(1)
User playlists	Map<String, PlaylistState>	HashMap	Names are unique only inside this owner; identical names under different owners remain separate	create / owner-local lookup: expected O(1)
PlaylistState songs	List<Song>	ArrayList	Preserves insertion order and permits duplicate songs; list remove finds the first match	append: amortized O(1); remove-first / snapshot: O(P)
PlaylistState collaborators	Set<String>	HashSet	Permission membership is unique; the owner is checked separately	grant / membership: expected O(1)

Complexity notation and assumptions

C is catalog size; **k** is a search result size; **P** is playlist length. Hash operations have expected, not worst-case, constant cost, and dynamic-array append is amortized. String hashing/case folding also costs time proportional to the input text length. Index lists store references to the same immutable Song objects, not copies of each song.

Each new song is validated for duplicate identity before any catalog/index insertion. No public method returns these internal maps, lists, or their mutable views. Catalog size contributes O(C) references in the ID map and O(C) references in each of the two secondary indexes.

Collections and indexes

Part 2 of 2 | uniqueness, listening state, ranking, and client snapshots

Field or index	Declared type	Concrete collection used	Reason for the choice	Expected complexity of important operations
User followedPlaylists	Set<PlaylistState>	LinkedHashSet	Unique identity membership; stable internal playlist instances distinguish same-name playlists under different owners	follow / duplicate check: expected O(1); snapshots: O(D + S)
User favorites	Set<Song>	LinkedHashSet	Songs compare by immutable, case-sensitive ID; repeat additions are idempotent	add / remove / contains: expected O(1); snapshot / genre extraction: O(F)
User history	Deque<Song>	ArrayDeque	Newest event at the front; oldest at the back; at most 50 play events including repeats	record / evict: amortized O(1); recent query: O(H), where H <= 50
User everPlayed	Set<String>	HashSet	Lifetime exclusion must survive history eviction; IDs avoid retaining additional song objects	insert / membership: expected O(1); storage: O(E)
SongRanking playCounts	Map<String, Long>	HashMap	Global counts for played songs only; absent entries count as zero; long provides ample count range	increment / lookup: expected O(1); storage: O(G)
SongRanking select selected (local)	PriorityQueue<Song>	PriorityQueue	Worst retained song at the head; only up to requested n candidates occupy the heap	offer / replacement: O(log(q + 1)); head: O(1); q <= min(n, C)
Playlist songs and public results	List<Song>, List<Playlist>, Set<Song>	JDK immutable snapshots via List.copyOf / Set.copyOf / List.of	Client writes cannot change service data; snapshots do not update when internal state changes	creation: O(result size); list read: O(1); favorite-set membership: expected O(1)

Additional local collections

Recent-query duplicate filtering uses a local HashSet<Song> and ArrayList<Song>, each bounded by 50 entries. Favorite-genre extraction uses a local HashSet<String>. Ranked results use an ArrayList before making an immutable snapshot. Recommendation candidates traverse indexed buckets with an iterator, avoiding an extra list containing every candidate.

D is followed playlist count; **S** is the sum of their song counts; **F** is favorite count; **E** is lifetime unique played count; **G** is global unique played count. LinkedHashSet is used for stable iteration where useful, although followed/favorite order is not required by the API.

Behavior and algorithms

Validation, permissions, bounded history, and deterministic ranking

Validation and identity

Validation.require rejects null with IllegalArgumentException. Validation.nonnegative rejects negative n. Unknown users and playlists are errors; unknown songs are errors except getSong, which returns null. Zero-size queries still validate their user. IDs and usernames remain case-sensitive. Blank song IDs retain the starter rejection; positive durations are enforced in Song. The specification imposes no nonblank rule for titles, artists, genres, usernames, or playlist names.

Song is final and immutable, and equals/hashCode use only ID. Catalog identity cannot become inconsistent after insertion. Artist, genre, and title matching use the same Unicode simple case fold: Character.toLowerCase(Character.toUpperCase(codePoint)). This is independent of the default locale, handles final sigma and dotted I consistently with Java simple case-insensitive equality, and avoids multi-character case expansions.

Permissions and encapsulation

The service resolves the acting user, owner, playlist, and song before changing state. The owner or a stored collaborator can add/remove songs. Only the owner grants collaboration; adding the owner as a collaborator is rejected, and duplicate grants do not change membership. Follows must target another owner and never grant editing rights. Public Playlist objects are final snapshots with no mutators, and their song lists are immutable. Re-fetching followed playlists returns current contents.

History and lifetime exclusions

On a play, User evicts the oldest deque event if 50 entries already exist, prepends the new event, and records the song ID in everPlayed. SongRanking increments the global count. getRecentlyPlayed walks newest to oldest and adds only the first occurrence of each song until n unique songs have been collected. Repeats occupy event slots and therefore can evict older songs even when fewer than 50 unique songs remain. Lifetime played IDs are never evicted.

Shared ranking and recommendation selection

One comparator orders global count descending, title with String.CASE_INSENSITIVE_ORDER, then ID in natural case-sensitive order. Zero-play songs participate. SongRanking.select uses the reverse comparator in a bounded heap: append while under capacity; otherwise replace the worst retained candidate only when the new candidate ranks higher. Sort the retained songs with the original comparator before returning the immutable result.

Top songs traverse the catalog. Recommendations extract distinct favorite genres, traverse only their indexed buckets, and exclude favorites and lifetime plays. A song belongs to one genre bucket, so candidate duplication is avoided even when multiple favorites share a genre. With no favorite genres, return an empty list.

Time and space tradeoffs

For M traversed candidates and q = min(n, C), selection costs $O(M \log(q + 1) + q \log(q + 1))$ time and $O(q)$ heap/result space. Extracting favorite genres adds $O(F)$ time and $O(g)$ space for g distinct favorite genres. Artist/genre results cost $O(k)$, which is necessary to return k songs. Title scans cost $O(C)$ for fixed-length text and also include text-folding/matching costs. Playlist snapshots cost $O(P)$. The service is in-memory and single-threaded, matching the starter and assignment; persistence and concurrent access are outside the required API.